

Welcome to The Carpentries Etherpad!

This pad is synchronized as you type, so that everyone viewing this page sees the same text. This allows you to collaborate seamlessly on documents.

Use of this service is restricted to members of The Carpentries community; this is not for general purpose use (for that, try <https://etherpad.wikimedia.org>).

Users are expected to follow our code of conduct: <https://docs.carpentries.org/policies/coc>

All content is publicly available under the Creative Commons Attribution License:
<https://creativecommons.org/licenses/by/4.0/>

Foundations of Astronomical Data Science Collaborative Notes Document

Links:

Workshop website: <https://abostroem.github.io/2026-01-03-aas/>

ADQL: <https://www.ivoa.net/documents/ADQL/>

Carpentries blog post on GenAI: <https://carpentries.org/blog/2025/08/genai-curriculum-discussions/>

Episode 1

Navigate to the "student_download" directory.

Activate the python environment -- conda activate AstronomicalData

Open jupyter notebook.

Step 1: Get Gaia data from the Gaia database. Type the following on the jupyter notebook cell and run the cell:

```
from astroquery.gaia import Gaia
```

```
tables = Gaia.load_tables(only_names=True)
```

```
for table in tables:
```

- `print(table.name)`

To get the metadata of a single table just to check before downloading the massive table-----

```
table_metadata = Gaia.load_table('gaiadr2.gaia_source')
```

table_metadata

print(table_metadata)

To get the info in the columns of the table -----

for column in table_metadata.columns:

- **print(column.name)**

Exercise 1: use load_table to get metadata of this table and get the info in the columns of this table ---

gaiadr2.panstarrs1_original_valid

ANSWER:

panstarrs_metadata = Gaia.load_table('gaiadr2.panstarrs1_original_valid')

print(panstarrs_metadata)

for column in panstarrs_metadata.columns:

- **print(column.name)**

Now we will type out an ADQL query to retrieve data from the Gaia Database -----

```
query1 = """SELECT  
TOP 10  
source_id, ra, dec, parallax  
FROM gaiadr2.gaia_source  
"""
```

To run this query, we use a Gaia object -----

job1 = Gaia.launch_job(query1)

job1

To get the results of the job ----

results1 = job1.get_results()

type(results1)

To display the Astropy table and datatypes in the columns, type -----

results1

Exercise 2: Read the documentation of this table

(https://gea.esac.esa.int/archive/documentation/GDR2/Gaia_archive/chap_datamodel/sec_dm_main_tables/ssec_dm_gaia_source.html) and choose a column that looks interesting to you. Add

the column name to the query and run it again. What are the units of the column you selected? What is its data type? ---

ANSWER (example -- radial_velocity column):

```
query1_with_rv = """SELECT
TOP 10
source_id, ra, dec, parallax, radial_velocity
FROM gaiadr2.gaia_source
"""
```

```
job1_with_rv = Gaia.launch_job(query1_with_rv)
results1_with_rv = job1_with_rv.get_results()
```

```
results1_with_rv
```

This is a synchronous job -- limited to only 2000 rows. We need asynchronous queries to run jobs with many more rows. We will now show an example of an asynchronous query with a condition on parallax.

```
query2 = """SELECT
TOP 3000
source_id, ra, dec, pmra, pmdec, parallax,
FROM gaiadr2.gaia_source
WHERE parallax < 1
"""
```

To run the asynchronous job (slightly slower than synchronous, but useful for large datasets)-----

```
job2 = Gaia.launch_job_async(query2)
job2
```

To get the results of this job ----

```
results2 = job2.get_results()
results2
```

Now, try to redo the query by changing the conditions ----

```
query2 = """SELECT
TOP 3000
source_id, ra, dec, pmra, pmdec, parallax,
FROM gaiadr2.gaia_source
WHERE parallax NOT == 1
"""
```

Exercise 3: Read about SQL operators here (https://www.w3schools.com/sql/sql_operators.asp) and then modify the previous query to select rows where bp_rp is between -0.75 and 2. ---

NOTE -- operators of SQL will work for ADQL as well.

```
query2_soln = """SELECT
TOP 10
source_id, ra, dec, pmra, pmdec, parallax,
FROM gaiadr2.gaia_source
WHERE parallax < 1
```

- AND bp_rp > -0.75 AND bp_rp < 2

```
"""
```

or we can use **BETWEEN**

```
query2_soln = """SELECT
TOP 10
source_id, ra, dec, pmra, pmdec, parallax,
FROM gaiadr2.gaia_source
WHERE parallax < 1
```

- AND bp_rp BETWEEN -0.75 AND 2

```
"""
```

NOTE -- Capitlization does not matter for queries, but it is usually a convention for readability.

All these are exaples of string literal queries. We can also write a python code to help us write a query so that human generated errors can be minimised. We use a query base and pass the column names as a variable. This can be done as follows -----

```
columns = 'source_id, ra, dec, pmra, pmdec, parallax'
```

```
query3_base = """SELECT
TOP 10
{columns}
FROM gaiadr2.gaia_source
WHERE parallax < 1
```

- AND bp_rp BETWEEN -0.75 AND 2

```
"""
```

Now, to assemble the query,

```
query3 = query3_base.format(columns=columns)
```

```
query3
```

```
print(query3)
```

Now, to run the query

```
job3 = Gaia.launch_job(query3)
print(job3)
```

To get the results,

```
results3 = job3.get_results()
results3
```

Exercise 4: This query always selects sources with parallax less than 1. But suppose you want to take that upper bound as an input. Modify query3 base to replace 1 with a format specifier like {max_parallax}. Now, when you call format, add a keyword argument that assigns a value to max_parallax, and confirm that the format specifier gets replaced with the value you provide.

ANSWER:

```
max_parallax = 0.5
```

```
query3_base = """SELECT
TOP 10
{columns}
FROM gaiadr2.gaia_source
WHERE parallax < {max_parallax}
      • AND bp_rp BETWEEN -0.75 AND 2
      """
```

```
query3_sol = query3_base_sol.format(columns=columns, max_parallax = max_parallax)
```

```
print(query3_sol)
```

• SESSION After the COFFEE BREAK

Episode 2

Using Astropy units ----

```
import astropy.units as u
```

To list all the units available ---

```
dir(u)
```

Examples -- to make an angle of 10 degrees ---

```
angle = 10 * u.degree
```

angle

Quantities provide a method called 'to' to various operations --- eg., to convert degree to arcmin

```
angle_arcmin = angle.to(u.arcmin)  
angle_arcmin
```

We can also add quantities --

```
angle + 30 * u.arcmin
```

```
angle + 5 * u.kg --- this results in an error because the dimensions are not compatible.
```

Exercise 5: Create a quantity that represents 5 arcminutes and assign it to a variable called *radius*. Then convert it to degrees.

ANSWER:

```
radius = 5 * u.arcmin
```

```
print(radius)
```

```
radius.to(u.degree)
```

Now, we want to write a query to select sources within a particular region of the sky.

```
cone_query = """SELECT  
TOP 10  
source_id  
FROM gaiadr2.gaia_source  
WHERE 1=CONTAINS(  
    • POINT(ra, dec),  
    • CIRCLE(88.8, 7.4, 0.0833333))  
"""
```

POINT here is location with co-ordinates in ICRS frame and CIRCLE is a region with co-ordinate pair and the radius of the region. CONTAINS makes sure that we only select points that fall within the specified region.

Now, to run the cone search query,

```
cone_job = Gaia.launch_job(cone_query)  
cone_job
```

To get the results of the job,

```
cone_results = cone_job.get_results()
cone_results
```

***Exercise 6:** When you are debugging queries like this, you can use TOP to limit the size of the results, but then you still don't know how big the results will be.*

An alternative is to use COUNT, which asks for the number of rows that would be selected, but it does not return them.

In the previous query, replace TOP 10 source_id with COUNT(source_id) and run the query again. How many stars has Gaia identified in the cone we searched?

ANSWER:

```
count_cone_query = """SELECT
COUNT(source_id)
FROM gaiadr2.gaia_source
WHERE 1=CONTAINS(
    POINT(ra, dec),
    CIRCLE(88.8, 7.4, 0.08333333333))
"""
```

```
count_cone_job=Gaia.launch_job(count_cone_query)
count_cone_results = count_cone_job.get_results()
count_cone_results
```

Our goal is ultimately to reproduce the figure in the GD-1 paper. But, first we want to test our query on a smaller dataset, i.e., get the data from a smaller rectangular region and once everything works, we expand the selection region to include all the stream stars of GD-1.

Before this, we want to use **co-ordinate transformations using built-in Astropy functions**. We will go from equatorial co-ords to Galactic co-ords and then finally to a specific frame of reference of the stream.

To do this, we first import the in-build astropy function:

```
from astropy.coordinates import SkyCoord
```

```
ra = 88.8 * u.degree
dec = 7.4 * u.degree
coord_icrs = SkyCoord(ra = ra, dec = dec, frame = 'icrs')
```

```
coord_icrs
```

SkyCoord also provides a *transform_to* function to transform co-ordinate systems. E.g., to transform from equatorial (ICRS) to galactic system -----

```
coord_galactic = coord_icrs.transform_to('galactic')
coord_galactic
```

Now, to transform from Galactic to another reference frame defined by Gala -- GD1Koposov10 --- this is a heliocentric ref frame defined by the orbit of the GD1 stream -- the co-ords in this frame are phi1 and phi2.

```
from gd1 import GD1Koposov10
```

```
gd1_frame = GD1Koposov10()  
gd1_frame
```

GD1Koposov10 is being imported from the student_download directory. So you need to be in that directory for the import to work.

Now, to get the coords in GD1 frame from the ICRS coords,

```
coord_gd1 = coord_icrs.transform_to(gd1_frame)  
coord_gd1
```

Here, we can just directly do the transformation from ICRS to GD1 frame. The transformation from ICRS to Galactic that we did above was just an example.

Exercise 7: Find the location of GD-1 in ICRS coordinates.

1. Create a SkyCoord object at $0^\circ, 0^\circ$ in the GD-1 frame.
2. Transform it to the ICRS frame.

Hint: Because ICRS is a standard frame, it is built into Astropy. You can specify it by name, icrs (as we did with galactic).

NOTE: The keyword arguments for the frame are found with:

```
gd1_frame.representation_component_names
```

ANSWER:

```
origin_gd1 = SkyCoord(0*u.degree, 0*u.degree, frame=gd1_frame)  
origin_gd1.transform_to('icrs')
```

• LUNCH BREAK

Episode 3

Now, we want to select a rectangular subset of stars in the gd1 stream and get their co-ordinates in the gd1 reference frame. We define the corners of the rectangular region in phi1 and phi2 co-ords.

```
phi1_min = -55 * u.degree
```



```

phi1_max = -45 * u.degree
phi2_min = -8 * u.degree
phi2_max = 4 * u.degree

```

Now, we define a function to make a rectangular region with the above co-ords as the corners of the rectangle.

```

def make_rectangle(x1, x2, y1, y2):
    """ Return the corners of the rectangle. """ ----- Maybe add a comment for later reference
    xs = [x1, x1, x2, x2, x1] ----- Here, we add the last co-ord to close the rectangle although it may
    not be needed
    ys = [y1, y2, y2, y1, y1]
    return xs, ys

```

These are going to contain the

```

phi1_rect, phi2_rect = make_rectangle(phi1_min, phi1_max, phi2_min, phi2_max)

```

```

corners = SkyCoord(phi1=phi1_rect, phi2 = phi2_rect, frame=gd1_frame)

```

```

corners

```

Transform these into ICRS because Gaia uses ICRS frame. But, it is easier to use the gd1_frame co-ords to select the rectangle and then transform it into ICRS to

```

corners_icrs = corners.transform_to('icrs')
corners_icrs

```

We just convert the corners to string

```

corners_list_str = corners_icrs.to_string()
corners_list_str

```

To finally parse these to ADQL, we need to format it accordingly (remove commas, quotations)--

```

corners_list_str = corners_icrs.to_string()
corners_list_str

```

```

corners_single_str.replace(' ', ', ')

```

We will now write a function to do this automatically instead of writing individual code lines----

```

def skycoord_to_string(skycoord):
    """Return the corners of a rectangle."""
    • corners_list_str = skycoord.to_string()
    • corners_single_str = ' '.join(corners_list_str)
    • return corners_single_str.replace(' ', ', ')

sky_point_list = skycoord_to_string(corners_icrs)

```

sky_point_list

Now, we are ready to retrieve the gd1 stars within a rectangular region from the Gaia database using a query. Let us now design the query.

```
query3_base = """SELECT
TOP 10
{columns}
FROM gaiadr2.gaia_source
WHERE parallax <1
    • AND bp_rp BETWEEN -0.75 AND 2
"""
```

Remember that the rectangular region in the gd1 frame becomes a polygon in the ICRS frame. So, to get sources within this polygon from Gaia database, the query is ---

```
polygon_top10query_base="""SELECT
TOP 10
{columns}
FROM gaiadr2.gaia_source
WHERE parallax <1
    • AND bp_rp BETWEEN -0.75 AND 2
    • AND 1 = CONTAINS(POINT(ra, dec), POLYGON({sky_point_list}))
"""
```

NOTE -- we only want those stars falling within polygon. So, the boolean for TRUE (==1) is used before CONTAINS.

Now, to write the query (we are using fetching Top10 to make sure everything is correct) ---

```
polygon_top10query = polygon_top10query_base.format(columns=columns,
sky_point_list=sky_point_list)
print(polygon_top10query)
```

Now, to launch the job ---

```
polygon_top10query_job = Gaia.launch_job_async(polygon_top10query)
print(polygon_top10query_job)
```

Now, to obtain the results ----

```
polygon_top10query_results = polygon_top10query_job.get_results()
polygon_top10query_results
```

Once we verify that the query works to fetch the top 10 rows, we now try to fetch all the sources within the polygon region ---

```

polygon_query_base = """SELECT
{columns}
FROM gaiadr2.gaia_source
WHERE parallax <1
    • AND bp_rp BETWEEN -0/75 AND 2
    • AND 1 = CONTAINS(POINT(ra, dec), POLYGON({sky_point_list}))
"""

```

Now, write the query using the python function:

```

polygon_query = polygon_query_base.format(columns=columns, sky_point_list=sky_point_list)
print(polygon_query)

```

To run the full job,

```

polygon_job = Gaia.launch_job_async(polygon_query)
print(polygon_job)

```

To get the results of the job, ----

```

polygon_results = polygon_job.get_results()

```

To save the resulting data into a fits file, -----

```

filename = 'gd1_results.fits'
polygon_results.write(filename, overwrite=True)

```

To get the size of the data file retrieved, -----

```

from os.path import getsize

```

```

MB = 1024 * 1024
getsize(filename) / MB

```

Now, to plot and visualize the results, we can do various tasks like ----

```

polygon_results.info()

```

To get the column names of the astropy table, -----

```

polygon_results.colnames

```

To get the values in individual columns, -----

polygon_results['ra']

To get values of the first row, ----

polygon_results[0]

To get the data type of the value in the particular row and column, ----

polygon_results['ra'] [0]

Now, to visualize the data, we import matplotlib ----

import matplotlib.pyplot as plt
%matplotlib inline

In recent versions of Jupyter, plots appear “inline”; that is, they are part of the notebook. In some older versions, plots appear in a new window. If your plots appear in a new window, you might want to run the following Jupyter magic command in a notebook cell: **%matplotlib inline**

Now, to visualize the data within the polygon that we extracted in the astropy table, ---

```
x = polygon_results['ra']
y = polygon_results['dec']
plt.plot(x, y, 'ko') -- here 'k' stands for the color black and 'o' for circle. You can also define explicitly
color = 'k'
```

```
plt.xlabel('RA (deg ICRS)')
plt.ylabel('Dec (deg ICRS)')
```

We can now see that the polygon is kind of curved in the ICRS frame as opposed to the rectangular region we defined in the gd1 frame.

Exercise 8: In the call to `plt.plot`, use the keyword argument `markersize` to make the markers smaller.

Then add the keyword argument `alpha` to make the markers partly transparent.

Adjust these arguments until you think the figure shows the data most clearly.

Note: Once you have made these changes, you might notice that the figure shows stripes with lower density of stars. These stripes are caused by the way Gaia scans the sky, which you can read about [here](#). The dataset we are using, Gaia Data Release 2, covers 22 months of observations; during this time, some parts of the sky were scanned more than others.

ANSWER:

```
x = polygon_results['ra']
y = polygon_results['dec']
plt.plot(x, y, 'ko', markersize=0.1, alpha=0.1)
plt.xlabel('ra (degree ICRS)')plt.ylabel('dec (degree ICRS)')
```

Remember that we selected data from a rectangle of coordinates in the GD-1 frame, then transformed them to ICRS when we constructed the query. The coordinates in the query results are in ICRS. To plot them, we will transform them back to the GD-1 frame; that way, the axes of the figure are aligned with the orbit of GD-1, which is useful for two reasons:

- By transforming the coordinates, we can identify stars that are likely to be in GD-1 by selecting stars near the centerline of the stream, where b is close to 0.
- By transforming the proper motions, we can identify stars with non-zero proper motion along the l axis, which are likely to be part of GD-1.

To do the transformation, we will put the results into a SkyCoord object. In a previous episode, we created a SkyCoord object like this:

```
skycoord = SkyCoord(ra = polygon_results['ra'], dec=polygon_results['dec'])
```

Notice that we did not specify the reference frame. That is because when using ra and dec in SkyCoord, the ICRS frame is assumed by default.

The SkyCoord object can keep track not just of location, but also proper motions. This means that we can initialize a SkyCoord object with location and proper motions, then use all of these quantities together to transform into the GD-1 frame.

Now we are going to do something similar, but now we will take advantage of the SkyCoord object's capacity to include and track space motion information in addition to ra and dec. We will now also include:

- pmra and pmdec, which are proper motion in the ICRS frame, and
- distance and radial_velocity, which are important for the reflex correction and will be discussed in that section.

```
distance = 8 * u.kpc
```

```
radial_velocity = 0 * u.km/u.sec
```

```
skycoord = SkyCoord(ra = polygon_results['ra'],
```

- dec=polygon_results['dec'],**
- pm_ra_cosdec = polygon_results['pmra'],**
- pm_dec = polygon_results['pmdec'],**
- distance = distance,**
- radial_velocity = radial_velocity**
-)**

To transform to the gd1 frame now, ----

```
transformed = skycoord.transform_to(gd1_frame)
```

Reflex correction: --- Now, we need to correct the proper motion to the the effect of the motion of our solar system around the Galactic center.

When we created skycoord, we provided constant values for distance and radial_velocity rather than measurements from Gaia.

That might seem like a strange thing to do, but here is the motivation:

- Because the stars in GD-1 are so far away, parallaxes measured by Gaia are negligible, making the

distance estimates unreliable.

- So we replace them with our current best estimate of the mean distance to GD-1, about 8 kpc. See Koposov, Rix, and Hogg, 2010.
- For the other stars in the table, this distance estimate will be inaccurate, so reflex correction will not be correct. But that should have only a small effect on our ability to identify stars with the proper motion we expect for GD-1.
- The measurement of radial velocity has no effect on the correction for proper motion, but we have to provide a value to avoid errors in the reflex correction calculation. So we provide 0 as an arbitrary place-keeper.

With this preparation, we can use `reflex_correct` from Gala ([documentation here](#)) to correct for the motion of the solar system which we have included as a stand alone file in the `student_download`. ----

```
from relex import reflex_correct
```

```
skycoord_gd1 = reflex_correct(transformed)
```

Now, we have reflex corrected proper motions in the `gd1` frame. We can visualise everything in the `gd1` frame:

```
x = skycoord_gd1.phi1  
y = skycoord_gd1.phi2  
plt.plot(x, y, 'ko', markersize=0.1, alpha=0.1)  
plt.xlabel('phi1 (degree GD1)')  
plt.ylabel('phi2 (degree GD1)')
```

• BREAK

Episode 4

```
type(polygon_results)
```

```
type(skycoord_gd1)
```

These are different data types, so it is convenient to have them as similar data type. For this, we import the `skycoord` values into new columns in the `astropy` table.

```
polygon_results['phi1'] = skycoord_gd1.phi1  
polygon_results['phi2'] = skycoord_gd1.phi2  
polygon_results.info()
```

Adding on to this, we can also import the proper motion results as new columns in the `astropy` table ---

```
polygon_results['pm_phi1'] = skycoord_gd1.pm_phi1_cosphi2  
polygon_results['pm_phi2'] = skycoord_gd1.pm_phi2  
polygon_results.info()
```

How is pandas dataframe different from Astropy table? There are a lot of similarities between both. But, one maybe be preferred over the other in certain cases. Pandas dataframes perform calculations faster compared to Astropy tables, especially when the data size is very large. Also, pandas dataframe is more widely used (outside of Academia), so it is good to have experience using pandas dataframes.

It is common to use "_df" to denote pandas dataframes

```
import pandas as pd
```

```
results_df = polygon_results.to_pandas()
```

results_df.shape ----- gives rows, columns. Shape is an attribute, so there is no paranthesis at the end

results_df.head() ---- fetches the first 5 rows. Head is a function applied on the df, so needs ()

results_df.tail() --- fetches last couple of rows

Before we go any further, we will take all of the steps that we have done and consolidate them into a single function that we can use to take the coordinates and proper motion that we get as an AstropyTable from our Gaia query, add columns representing the reflex corrected GD-1 coordinates and proper motions, and transform it into a Pandas DataFrame. This is a general function that we will use multiple times as we build different queries so we want to write it once and then call the function rather than having to copy and paste the code over and over again.

```
def make_dataframe(table):
```

```
    """Transform coordinates from ICRS to GD-1 frame.
```

```
    table: Astropy Table
```

```
    returns: Pandas DataFrame
```

```
    """
```

```
    #Create a SkyCoord object with the coordinates and proper motions in the input table
```

```
    skycoord = SkyCoord(
```

- **ra=table['ra'], dec=table['dec'],**
- **pm_ra_cosdec=table['pmra'],**
- **pm_dec=table['pmdec'], distance=8*u.kpc,**
- **radial_velocity=0*u.km/u.s**
- **)**

```
    # Define the GD-1 reference frame
```

```
    gd1_frame = GD1Koposov10()
```

```
    # Transform input coordinates to the GD-1 reference frame
```

```
    transformed = skycoord.transform_to(gd1_frame)
```

```
    # Correct GD-1 coordinates for solar system motion around galactic center
```

```
    skycoord_gd1 = reflex_correct(transformed)
```

```
    #Add GD-1 reference frame columns for coordinates and proper motions
```

```
    table['phi1'] = skycoord_gd1.phi1
```

```

table['phi2'] = skycoord_gd1.phi2
table['pm_phi1'] = skycoord_gd1.pm_phi1_cosp2
table['pm_phi2'] = skycoord_gd1.pm_phi2

```

```

# Create DataFrame
df = table.to_pandas()

```

```

return df

```

Now, we can utilize this function to run the query and get the results into a pandas dataframe

```

results_df = make_dataframe(polygon_results)

```

Now, to save the pandas dataframe in a binary format which is easy to handle and also save metadata related to the table -- 'HDF5' -- this format has multiple advantages. But, we can also save the pandas dataframe in other formats like '.csv' if that is more convenient.

```

filename = 'gd1_data.hdf'
results_df.to_hdf(filename, key='results_df', mode='w')

```

```

results_df.describe(). -- to get more details like mean, std, max, min etc.

```

NOTE: The information of units of the columns will be lost when you convert Astropy tables into pandas dataframe. The work around maybe to name the columns of pandas dataframe to include the units.

Exercise 9: Review the summary statistics in this table.

- Do the values make sense based on what you know about the context?
- Do you see any values that seem problematic, or evidence of other data issues?

Now, we are ready to replicate one of the panels of the plot in the Price-Whelan, Bonaca paper --- Plotting proper motion of the stars and selecting those belonging to the GD1 frame.

```

x = results_df['pm_phi1']
y = results_df['pm_phi2']
plt.plot(x, y, 'ko', markersize=0.1, alpha=0.1)
plt.xlabel('Proper motion phi1 (mas/yr GD1 frame)')
plt.ylabel('Proper motion phi2 (mas/yr GD1 frame)')

```

To visualize better and zoom into the extreme values in order to find those sources belonging to the GD1 stream,

```

x = results_df['pm_phi1']
y = results_df['pm_phi2']
plt.plot(x, y, 'ko', markersize=0.1, alpha=0.1)
plt.xlabel('Proper motion phi1 (mas/yr GD1 frame)')

```



```
plt.ylabel('Proper motion phi2 (mas/yr GD1 frame)')
plt.xlim(-12, 8)
plt.ylim(-10, 10)
```

There is a hint of an overdense region near (-7.5, 0), but if you did not know where to look, you would miss it.

To see the cluster more clearly, we need a sample that contains a higher proportion of stars in GD-1. We will do that by selecting stars close to the centerline.

Stars near the $\text{phi2} = 0$ line have the highest probability of being in GD-1.

To select them, we will use a “Boolean mask”. We will start by selecting the phi2 column from the DataFrame:

```
phi2 = results_df['phi2']
type(phi2)
```

The result is a Series, which is the structure Pandas uses to represent columns.

We can use a comparison operator, $>$, to compare the values in a Series to a constant.

```
phi2_min = -1.0 * u.degree
phi2_max = 1.0 * u.degree
mask = (phi2 > phi2_min)
type(mask)
```

The result is a Series of Boolean values, that is, True and False.

```
mask.head()
```

To select values that fall between phi2_min and phi2_max, we will use the $\&$ operator, which computes “logical AND”. The result is true where elements from both Boolean Series are true.

```
mask = (phi2 > phi2_min) & (phi2 < phi2_max)
```

NOTE: Python’s logical operators (and, or, and not) do not work with NumPy or Pandas. Both libraries use the bitwise operators ($\&$, $|$, and $\sim$) to do elementwise logical operations. Also, we need the parentheses around the conditions; otherwise the order of operations is incorrect.

The sum of a Boolean Series is the number of True values, so we can use sum to see how many stars are in the selected region.

```
mask.sum()
```

A Boolean Series is sometimes called a “mask” because we can use it to mask out some of the rows in a DataFrame and select the rest, like this:

```
centerline_df = results_df[mask]
```

type(centerline_df)

centerline_df is a DataFrame that contains only the rows from results_df that correspond to True values in mask. So it contains the stars near the centerline of GD1.

We can use len to see how many rows are in centerline_df:

len(centerline_df)

To check what fraction of the rows we have selected ----

len(centerline_df) / len(results_df)

We will now write a function to make the plot of proper motion of this subset in the GD1 frame ---

def plot_proper_motion(df):

- *"""Plot proper motion.*
- *df: DataFrame with `pm\_phi1` and `pm\_phi2`*
- *"""*
- `x = df['pm_phi1']`
- `y = df['pm_phi2']`
- `plt.plot(x, y, 'ko', markersize=0.3, alpha=0.3)`
- `plt.xlabel('Proper motion phi1 (mas/yr)')`
- `plt.ylabel('Proper motion phi2 (mas/yr)')`
- `plt.xlim(-12, 8)`
- `plt.ylim(-10, 10)`

And we can call it like this:

plot_proper_motion(centerline_df)

Now we can see more clearly that there is a cluster near (-7.5, 0).

The next step is to select stars in the “overdense” region of proper motion, which are candidates to be in GD-1.

In the original paper, Price-Whelan and Bonaca used a polygon to cover this region. But, we will use a simple rectangle for now, but in a later lesson we will see how to select a polygonal region as well. Here are bounds on proper motion we chose by eye:

pm1_min = -8.9

pm1_max = -6.9

pm2_min = -2.2

pm2_max = 1.0

To draw these bounds, we will use the make_rectangle function we wrote in episode 2 to make two lists containing the coordinates of the corners of the rectangle.

```
pm1_rect, pm2_rect = make_rectangle(pm1_min, pm1_max, pm2_min,  
pm2_max)
```

Now, to make the plot with the bounds that we chose ----

```
plot_proper_motion(centerline_df)  
plt.plot(pm1_rect, pm2_rect, '-')
```

Now that we have identified the bounds of the cluster in proper motion, we will use it to select rows from `results_df`.

We will use the following function, which uses Pandas operators to make a mask that selects rows where series falls between low and high.

```
def between(series, low, high):  
    """Check whether values are between `low` and `high`."""  
    return (series > low) & (series < high)
```

The following mask selects stars with proper motion in the region we chose.

```
pm1 = results_df['pm_phi1']  
pm2 = results_df['pm_phi2']  
pm_mask = (between(pm1, pm1_min, pm1_max) & between(pm2, pm2_min, pm2_max))
```

Again, the sum of a Boolean series is the number of TRUE values.

```
pm_mask.sum()
```

Now we can use this mask to select rows from `results_df`.

```
selected_df = results_df[pm_mask]  
len(selected_df)
```

These are the stars we think are likely to be in GD-1. We can inspect these stars, plotting their coordinates (not their proper motion).

```
x = selected_df['phi1']  
y = selected_df['phi2']  
plt.plot(x, y, 'ko', markersize=1, alpha=1)  
plt.xlabel('phi1 (degree GD1)')  
plt.ylabel('phi2 (degree GD1)')
```

To clean up the plot a little bit we can add two new Matplotlib commands:

- `axis` with the parameter `equal` sets up the axes so a unit is the same size along the x and y axes.
- `title` puts the input string as a title at the top of the plot. The `fontsize` keyword sets the `fontsize` to be medium, a little smaller than the default large.

```
plt.plot(x, y, 'ko', markersize=0.3, alpha=0.3)
plt.xlabel('phi1 [deg]')
plt.ylabel('phi2 [deg]')
plt.title('Proper motion selection', fontsize='medium')
plt.axis('equal')
```

Now, we will put the code we wrote to make one of the panel figures into a function that we will use in future episodes to recreate this entire plot with a single line of code.

def plot_pm_selection(df):

"""Plot in GD-1 spatial coordinates the location of the stars selected by proper motion """

- `x = df['phi1']`
- `y = df['phi2']`
- `plt.plot(x, y, 'ko', markersize=0.3, alpha=0.3)`
- `plt.xlabel('phi1 [deg]')`
- `plt.ylabel('phi2 [deg]')`
- `plt.title('Proper motion selection', fontsize='medium')`
- `plt.axis('equal')`

We can now apply this function to make the plot ----

plot_pm_selection(selected_df)

To save the dataframe: ----

```
filename = 'gd1_data.hdf'
selected_df.to_hdf(filename, key='selected_df')
```

Exercise 9: We are going to need `centerline_df` later as well. Write a line of code to add it as a second Dataset in the HDF5 file.

Hint: Since the file already exists, you should not use `mode='w'`.

ANSWER:

centerline_df.to_hdf(filename, key='centerline_df')

We can use `getsize` to confirm that the file exists and check the size

getsize(filename) / MB

If you forget what the names of the Datasets in the file are, you can read them back like this:

with pd.HDFStore(filename) as hdf:

- `print(hdf.keys())`

We use a with statement here to open the file before the print statement and (automatically) close it after.

• **END of DAY 1**

Set up a gaia account: <https://www.cosmos.esa.int/web/gaia-users/register>